PROJECT REPORT

RentEase

A full-stack property rental platform for modern travellers

RentEase lets property owners publish rental listings with photos, pricing and location, and lets travellers discover them through category filters, destination search, interactive maps and star-rated reviews. It is a server-rendered MVC web application built on Node.js, Express and MongoDB.

Node.js | Express 5 | MongoDB | EJS | Cloudinary | Mapbox

Repository	github.com/DivyaAsthana8898/RentEase-main
Application version	1.2.0 (package.json)
Runtime	Node.js 22.14.0
License	MIT
Report date	1 November 2026

CONTENTS

1 Abstract and Objectives 2
2 Technology Stack and Setup 3
3 System Architecture 4
4 Database Design 5
5 Routes, Security and Validation 6
6 User Interface and Functional Testing 7
7 Review Findings, Future Scope and Conclusion 8

1. Abstract and Objectives

RentEase is a server-rendered, full-stack web application for listing, discovering and reviewing rental properties. Owners publish a listing with photos, a nightly price, a category and a location. Travellers browse a card catalogue, filter it by category (beaches, mountains, pools and so on), search by destination, see each property on an interactive map and read or write star-rated reviews. The system follows the MVC pattern on Node.js and Express 5, stores its data in MongoDB Atlas through Mongoose, keeps uploaded photos on Cloudinary, converts addresses to coordinates with the Mapbox Geocoding API, and protects every write operation with Passport.js session authentication and ownership-based authorization.

This report describes the problem, the architecture and data model, the main implementation modules, the user interface, observations from a code review and the directions in which the project can grow.

Problem statement

Travellers and renters often compare homes across scattered sources with little structured feedback, while small property owners need a lightweight way to publish a place, show where it is and collect reviews. RentEase provides one place for both sides: a searchable, map-aware catalogue with accountable, logged-in contributors.

Objectives

- Let authenticated users create, edit and delete property listings, with photos stored in the cloud.
- Make discovery quick through category filters, destination search and a responsive card grid.
- Show each property on an interactive map by geocoding the address entered by the owner.
- Collect trust signals through 1 to 5 star ratings and written comments tied to a logged-in author.
- Enforce who may change what: only a listing's owner can edit or delete it, only a review's author can remove it.
- Keep the codebase organised (MVC, reusable middleware, centralised error handling) so features can be added safely.

Project at a glance

Name / version	RentEase, 1.2.0	**Hosted URL (page metadata)**	rentease-lxfu.onrender.com
Stack	MERN-style: Node, Express, MongoDB, server-side EJS	**Source credit**	package.json and README credit Vinay Tambey (upstream VinayNoogler000/RentEase)
Codebase	52 files, about 2,800 lines of JS, EJS and CSS	**Seed data**	29 sample listings, 9 categories, 16 countries

Feature scope

Feature	Status	Where it lives
Sign up, log in, log out (auto-login after sign-up)	Implemented	controllers/user.js, passport-local-mongoose
Create, edit, delete listings with photo upload	Implemented	controllers/listing.js, Multer + Cloudinary
Browse all listings as a responsive card grid	Implemented	views/listings/index.ejs
Category filter (9 categories) and destination search	Implemented	filterListing, searchListing
Reviews with 1 to 5 star rating, delete own review	Implemented	controllers/review.js, rating.css
Interactive map per listing (geocoded address)	Implemented	utils/getGeoCoordinates.js, public/js/map.js
Price display with optional 18% tax toggle	Implemented	public/js/toggleTaxes.js
Responsive layout with off-canvas mobile navigation	Implemented	navbar.ejs, Bootstrap 5
Booking, payments, messaging, e-mail alerts, social login	Planned	Listed as to-do items in the README

2. Technology Stack and Setup

Versions below are taken from `package.json`; CDN assets are pinned in the layout template.

Layer	Technology	Version	Role in RentEase
Runtime	Node.js	22.14.0	Server-side JavaScript runtime (engines field)
Web framework	Express	^5.1.0	Routing, middleware pipeline, error handling
Templating	EJS, ejs-mate	^3.1.10, ^4.0.0	Server-rendered views, shared layout and partials
UI	Bootstrap, Font Awesome	5.3.6 (CDN), kit	Responsive grid, forms, off-canvas navbar, icons
Database	MongoDB Atlas, Mongoose	^8.15.1	Document store and object modelling
Authentication	Passport, passport-local, passport-local-mongoose	^0.7.0, ^1.0.0, ^8.0.0	Username/password strategy, salted password hashing
Sessions	express-session, connect-mongo	^1.18.1, ^5.1.0	Server-side sessions persisted in MongoDB
Flash messages	connect-flash	^0.1.1	One-time success and error alerts
Validation	Joi	^17.13.3	Request-body validation before controllers run
File upload	Multer, multer-storage-cloudinary	^2.0.2, ^4.0.0	Multipart parsing and streaming to Cloudinary
Media hosting	Cloudinary SDK	^2.7.0	Image storage and URL-based transformations
Maps	Mapbox SDK, Mapbox GL JS	^0.16.1, 3.13.0 (CDN)	Forward geocoding, interactive map and marker
Utilities	method-override, dotenv	^3.0.0, ^16.6.1	PUT/DELETE from HTML forms, environment configuration

Project structure and configuration

```
rentease/
|-- app.js            entry point, middleware, routes
|-- cloudConfig.js    Cloudinary + Multer storage
|-- schema.js         Joi schemas
|-- controllers/      listing, review, user
|-- middlewares/      isLoggedIn, isAuthorized,
|                     validateModel, saveRedirectUrl
|-- models/           listing, review, user
|-- routes/           listing, review, user
|-- utils/            ExpressError, wrapAsync,
|                     getGeoCoordinates
|-- init/             data.js (29 listings), index.js
|-- views/            layouts, includes, listings,
|                     users, error.ejs
`-- public/           css (10), js (4), preview.png
```

Variable (.env)	Purpose
PORT, NODE_ENV	Server port (default 8080), mode
SESSION_SECRET	Signs sessions, encrypts store
CLOUD_NAME	Cloudinary account name
CLOUD_API_KEY / SECRET	Cloudinary credentials
MAP_TOKEN	Mapbox access token
ATLASDB_URL	MongoDB connection string

Running the project locally

1. Clone the repository and run `npm install --force` (the command given in the README).
2. Create a `.env` file from `.env.sample` and fill in the MongoDB Atlas, Cloudinary and Mapbox values listed above.
3. Start the development server with `npm run dev` (runs `nodemon app.js`) and open `http://localhost:8080`; the root path redirects to `/listings`.
4. Optionally run `node init/index.js` to load the 29 sample listings. The script clears the listings collection and assigns every listing to a fixed owner id, so replace that id with a registered user's id first.

3. System Architecture

RentEase is a single Express application that renders HTML on the server. Every request passes through one global middleware pipeline, is dispatched to a router, is checked by route-level guards, and finally reaches a controller that talks to MongoDB and, when needed, to Cloudinary or Mapbox. Controllers hand data to EJS views, which are wrapped by one shared layout.

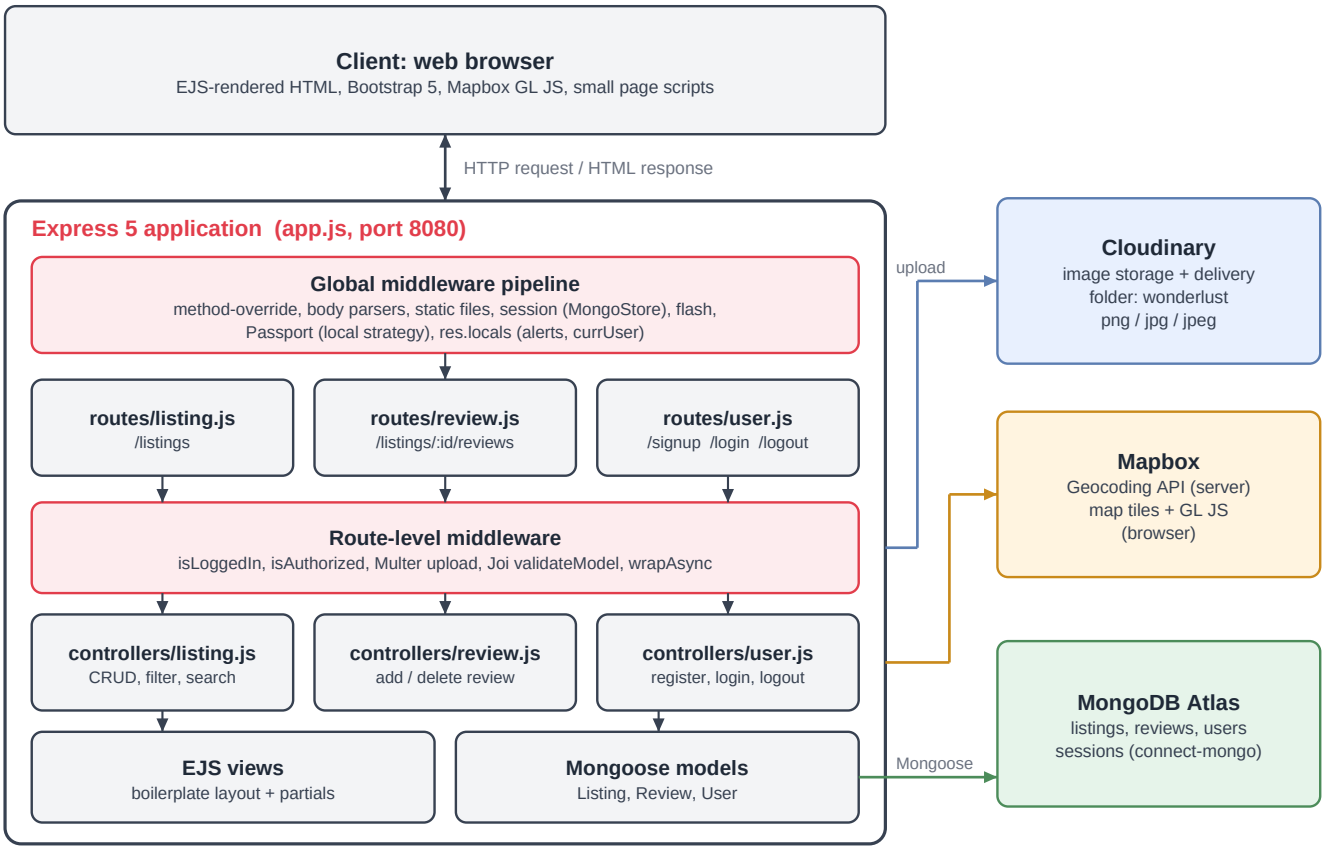


Figure 1. Component architecture. Uploaded images are served to the browser straight from Cloudinary URLs, and map tiles are fetched by Mapbox GL JS in the browser.

Request lifecycle: creating a listing

1. The browser submits the multipart form to `POST /listings`.
2. Global middleware restores the session from MongoDB, Passport deserialises the user, and flash alerts plus `currUser` are exposed to views.
3. `isLoggedIn` rejects anonymous users (and remembers the URL for after login); Multer streams the photo to the Cloudinary folder.
4. `validateModel(listingSchema)` checks the body with Joi and throws an `ExpressError(400)` on failure.
5. The controller geocodes *location, country* with Mapbox, builds the document with `owner = req.user._id`, saves it through Mongoose and redirects with a success flash.

Design decisions

Pattern	How it is used
MVC	Mongoose models hold schemas, EJS views render pages, controllers contain the route handlers; routers only wire paths to guards and controllers.
Middleware factory	`validateModel(schema)` and `isAuthorized('listing' | 'review')` return configured middleware, so one function protects several routes.
Async error wrapper	`wrapAsync` forwards rejected promises to `next(err)`, so a single global handler renders every error page.
Layout inheritance	ejs-mate wraps each page in `layouts/boilerplate.ejs`, with navbar, flash, filter and footer as partials.

4. Database Design

Data lives in three Mongoose-backed collections plus the session collection managed by connect-mongo. Relationships are modelled by reference rather than embedding, so reviews and owners can be queried and removed independently.

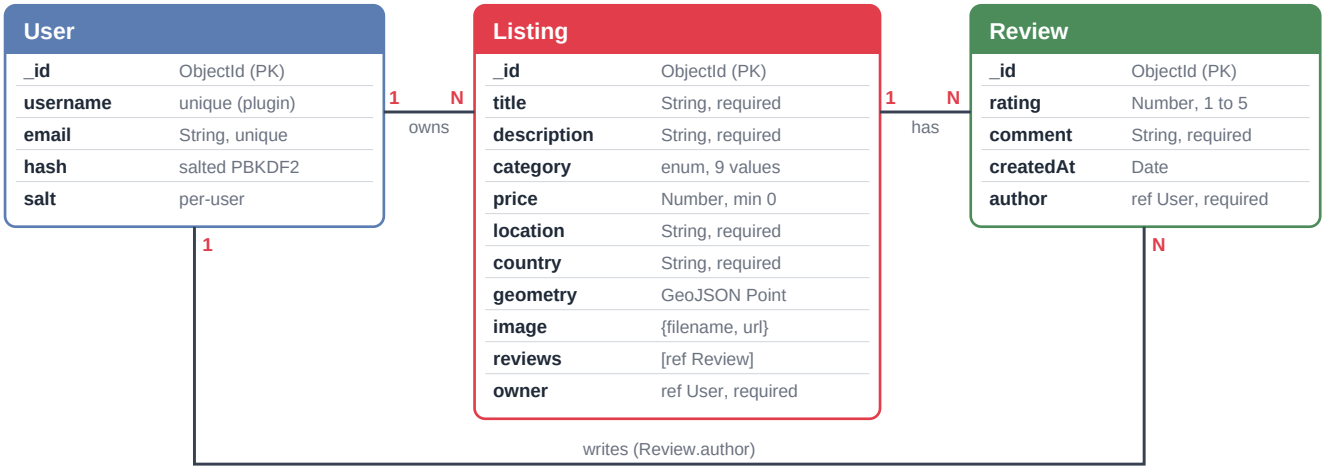


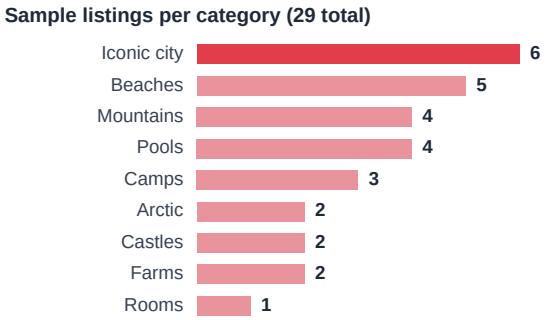
Figure 2. Entity-relationship view of the User, Listing and Review collections.

Schema details and validation rules

Collection	Key constraints	Notes
Listing	title, description, category, price, location, country and owner are required; price must be 0 or more; category is limited to 9 values: comfort-rooms, iconic-city, mountains, castles, pools, camps, farms, arctic, beaches.	`geometry` stores a GeoJSON Point returned by Mapbox. The image URL falls back to a default photo when none is supplied.
Review	rating is required, between 1 and 5; comment is required and trimmed; author references a User.	Linked to a listing through `Listing.reviews`; the star widget on the form posts the rating.
User	email is required and unique. The passport-local-mongoose plugin adds username, hash and salt and the register / authenticate helpers.	Passwords are never stored in plain text; only the salted hash is kept.
sessions	Created by connect-mongo with an encrypted payload (crypto secret) and a 24-hour `touchAfter` window.	Cookie is httpOnly with a 7-day lifetime.

Integrity and seed data

- **Cascade delete.** A Mongoose `post('findOneAndDelete')` hook removes every review of a deleted listing.
- **Review removal.** Deleting a review first `$pulls` its id from the listing, then deletes the document.
- **Read model.** The detail page populates reviews with their authors and the owner's id, username and email.
- **Seed script.** `init/index.js` geocodes each of the 29 sample listings (12 in the United States, the rest across 15 other countries) and inserts them in one batch.



Example listing document

```
{
  title: "Cozy Beachfront Cottage",  category: "beaches",  price: 1500,   // from init/data.js
  location: "Malibu",  country: "United States",
  geometry: { type: "Point", coordinates: [lng, lat] },        // Mapbox geocoder
  image: { filename: "...", url: "https://res.cloudinary.com/..." },
  reviews: [ ObjectId, ... ],   owner: ObjectId("...")         // refs to Review and User
}
```

5. Routes, Security and Validation

Method	Path	Guards	Purpose
GET	/	none	Redirect to /listings
GET	/listings	none	List all listings
GET	/listings/filter?category=	none	Listings of one category
GET	/listings/search?dest=	none	Match location or country
GET	/listings/new	login	New-listing form
POST	/listings	login, upload, Joi	Create listing (geocode, owner)
GET	/listings/:id	none	Detail page with reviews and map
GET	/listings/:id/edit	login, owner	Edit form with thumbnail
PUT	/listings/:id	login, owner, upload, Joi	Update listing (re-geocode)
DELETE	/listings/:id	login, owner	Delete listing and its reviews
POST	/listings/:id/reviews	login, Joi	Add review
DELETE	/listings/:id/reviews/:reviewId	login, author	Delete own review
GET, POST	/signup	Joi (POST)	Register, then auto-login
GET, POST	/login	Joi, Passport (POST)	Authenticate, redirect to origin
GET	/logout	login	End session
any	unmatched path	none	404 error page

Authentication and authorization

- **Credentials.** Passport's local strategy delegates to `User.authenticate()`; passport-local-mongoose generates a salt and stores a PBKDF2 hash, never the password.
- **Sessions.** Sessions are kept in MongoDB, referenced by an httpOnly cookie that lasts seven days. Users are serialised into the session with the plugin's helper methods.
- **Return to where you were.** `isLoggedIn` stores the requested URL; after login `loginUser` redirects there, trimming review and delete URLs back to the listing page.
- **Ownership checks.** `isAuthorized` compares the logged-in user's id with `listing.owner` or `review.author` and redirects with a flash message when they differ. The edit and delete buttons are also hidden from non-owners in the view.

Validation and error handling

- **Two layers.** Joi schemas in `schema.js` reject malformed listing, review and login bodies with HTTP 400 before a controller runs; Mongoose validators (required, min, max, enum) protect the database as a second line.
- **Client side.** Bootstrap's `needs-validation` script blocks empty submissions in the browser and shows inline feedback.
- **Central handler.** `ExpressError` carries a status code; a catch-all route turns unknown URLs into 404s, and the final error middleware renders `error.ejs` with the status and message.
- **Upload limits.** Cloudinary storage is configured for png, jpg and jpeg files in a dedicated folder.

6. User Interface and Functional Testing

The interface is a clean card layout with a fixed top bar (explore link, destination search, "Rent your home", login or logout), a horizontally scrollable strip of category filters and a switch that adds 18% tax to every displayed price.

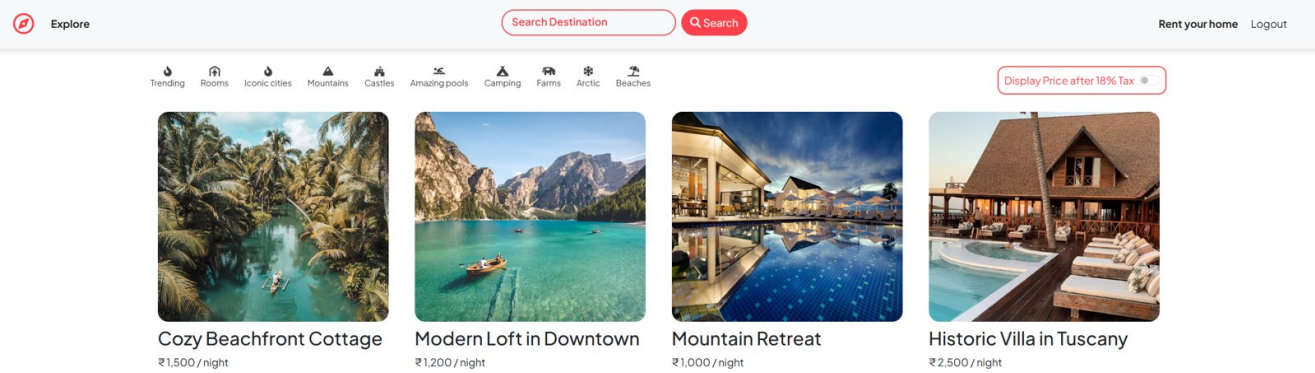


Figure 3. Home page: search bar, category filters, tax toggle and listing cards priced in Indian rupees per night.

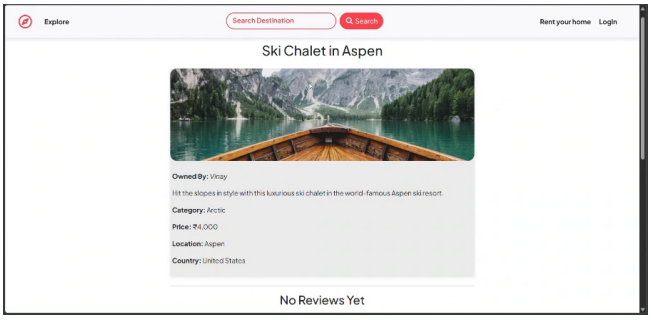


Figure 4. Listing detail: owner, description, category, price and location.

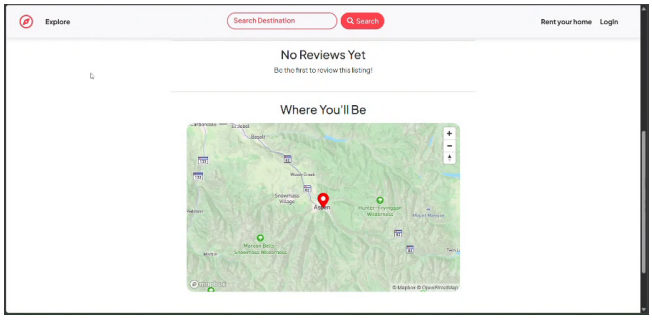


Figure 5. "Where You'll Be" map with a marker at the geocoded position.

How the main screens behave

- **Browse and filter.** Cards link to the detail page; the filter strip scrolls sideways with arrow buttons, and the tax switch recalculates every price on the page in the browser (price x 1.18).
- **Detail page.** Shows the owner, a star-rated review list and the map; edit and delete buttons appear only for the listing's owner, and the review form only for logged-in users.
- **Forms.** Sign-up, log-in, new-listing and edit-listing pages use Bootstrap validation; the edit page shows a 200 px thumbnail of the current photo and keeps it when no new file is chosen.

Functional test scenarios

Scenario	Expected result as implemented
Open the site root	Redirects to /listings and shows the card grid with image, title and price per night.
Choose a category filter or search "Aspen, United States"	Only matching listings are shown; a flash message appears when nothing matches.
Open /listings/new while logged out	Redirected to /login with a flash message; after login the user returns to the form.
Register with valid details	Account is created, the user is logged in automatically and lands on /listings.
Create a listing with a photo	Image goes to Cloudinary, the address is geocoded, the listing is saved with the current user as owner.
Edit or delete another user's listing	Blocked by isAuthorized; user is sent back to the listing with an error flash.
Submit a review with a star rating	Review is saved and linked; the page shows the author, stars and month of the review.
Delete a listing that has reviews	The listing and all of its reviews are removed.
Request an unknown URL	A 404 error page with the message "Page Not Found!".

Scenarios are derived from the implemented routes and middleware and from the bundled demo recording of the running site. The repository contains no automated test suite.

7. Review Findings, Future Scope and Conclusion

A read-through of the source produced the observations below. None blocks normal use, but several are worth fixing before a public launch.

Area	Observation	Suggested improvement
Script injection	show.ejs prints the listing as unescaped JSON inside a script block, and map.js places the title into popup HTML. A crafted title or description could run script.	Escape "<" in the serialised JSON or use data attributes; build the popup with text nodes.
Session setup	The cookie `expires` value is a number, the store error callback uses an undefined `err`, and sessions are created for every visitor.	Rely on maxAge, fix the handler signature, set saveUninitialized to false.
Hardening	No CSRF tokens, security headers or login rate limiting are configured.	Add CSRF protection, helmet and express-rate-limit.
Validation	Joi uses `allow()` for category, so only Mongoose enforces the list. The review default `Date.now()` runs once at start-up.	Use `valid()` and `default: Date.now`.
Geocoding	An unknown place makes getGeoCoordinates read a missing result and end in a 500 page.	Check for results and show a friendly form error.
Authorization	isAuthorized has no not-found handling, and the review delete button is shown to everyone (the server still blocks it).	Handle null records; render the button for the author only.
Search and filter	Search is an exact, case-sensitive match. The "Trending" filter has no matching category and always returns nothing.	Use case-insensitive or text search; define trending by rating or recency.
Housekeeping	Welcome message and Cloudinary folder still say "Wonderlust"; README links point to another repository; no start or test script, and nodemon is not declared.	Rename, fix links, add start script, dev dependencies and tests.

Future scope

- **Booking and payments:** availability calendar, date-conflict checks and a payment gateway (the map popup already promises exact location after booking).
- **Engagement:** real-time messaging between guest and host, e-mail notifications and Google / Facebook sign-in.
- **Richer listings:** multiple photos, average rating on cards with sort and filter by price or rating.
- **Performance:** indexes on category, location and country, pagination of the listing grid and image size variants.
- **Quality:** automated tests (for example Jest with Supertest) and a CI pipeline.

Conclusion

RentEase delivers the core of a rental marketplace in a compact, readable codebase: authenticated owners can publish geocoded, photo-rich listings, visitors can find them by category or destination, and logged-in users can leave ratings that are tied to their account. Its layered structure of models, controllers, routers and reusable middleware makes the planned booking, payment and messaging features straightforward to add, and the findings above give a short list of hardening steps for production use.

References

- Project repository: github.com/DivyaAsthana8898/RentEase-main (README, package.json and source files)
- Express: expressjs.com | EJS: ejs.co | Mongoose: mongoosejs.com | MongoDB Atlas: mongodb.com/atlas
- Passport.js: passportjs.org | Joi: joi.dev | Multer: github.com/expressjs/multer | Bootstrap 5: getbootstrap.com
- Cloudinary documentation: cloudinary.com/documentation | Mapbox Geocoding and GL JS: docs.mapbox.com